ASDD PRACTICAL EXAM

Complete Ubuntu Steps + Exact File Contents

Experiments 1–7 | Includes what to type inside every sample file

How to Use This Guide

- Commands are shown in grey code blocks. Type them in Ubuntu Terminal one at a time.
- Whenever nano opens, type or paste the exact content shown, then save with Ctrl+O, press Enter, and exit with Ctrl+X.
- Replace YOUR_USERNAME with your GitHub username and use your own email.
- For GitHub experiments, fork the specified repository in your browser before cloning.
- Jira experiments are completed in the browser; the remaining practicals mainly use Ubuntu Terminal.

Repository for Experiments 2 and 3: https://github.com/shruthi1722/node_test.git

Repository for Experiment 6: https://github.com/shruthi1722/terraform23

One-Time Setup: Git + GitHub Push Token

Install Git and configure your identity

```
sudo apt update
sudo apt install git -y
git --version
git config --global user.name "YOUR_USERNAME"
git config --global user.email "your-email@example.com"
git config --list
```

Create a GitHub Personal Access Token

1. Open https://github.com/settings/tokens and sign in.
2. Choose Tokens (classic) → Generate new token (classic).
3. Enter a note such as ASDD Practical and select an expiry.
4. Tick the repo scope and generate the token.
5. Copy the token immediately; GitHub will not show it again.

When git push asks for credentials, enter your GitHub username. At the password prompt, paste the token and press Enter. The token will not appear while you paste; that is normal.

Experiment 1: Jira Board and Sprint Management

This experiment is performed in the Jira browser interface; no Ubuntu file is required.

6. Open https://www.atlassian.com/software/jira and sign in.
7. Create a project and choose the Scrum template.
8. Use project name: Student Management System.
9. Open Backlog and create these six Stories (use the wording below).

Summary	Description / User story
Student Login	As a student, I want to log in so that I can access my account.
View Attendance	As a student, I want to view my attendance so that I

	can track my presence.
Manage Attendance	As a teacher, I want to manage attendance so that student records stay updated.
Manage Students	As an admin, I want to manage student details so that records remain accurate.
View Marks	As a student, I want to view my marks so that I can check my academic performance.
Manage Fees	As an admin, I want to manage fees so that payment records are maintained.

10. Create a sprint named Sprint 1 and move the six stories into it.
11. Start the sprint and set the duration requested by your examiner.
12. Move stories through To Do → In Progress → In Review → Done. If In Review is not available, use the workflow settings or an available review status.
13. Open each story and add a comment such as: "Work started", "Ready for review", or "Task completed".

Experiment 2: Jira and GitHub Integration

Browser: create a Jira Story named Implement Student Login. In the description enter: "Create a login feature for students." Note the actual Jira issue key shown after creation (example: SMS-1). Fork https://github.com/shruthi1722/node_test.git to your GitHub account.

Ubuntu commands

```
git clone https://github.com/YOUR_USERNAME/node_test.git
cd node_test
git checkout -b SMS-1-student-login
git branch
```

Replace SMS-1 with your actual Jira issue key.

Create login.txt and enter this exact content

```
nano login.txt
```

```
Student Login Feature
Jira Issue: SMS-1
Description: This file represents the student login task.
Status: Implementation started.
```

Save the file (Ctrl+O → Enter → Ctrl+X), then stage, commit and push:

```
git status
git add login.txt
git status
git commit -m "SMS-1 Implement student login"
git push -u origin SMS-1-student-login
```

Replace SMS-1 in the file and commit/branch names with your real Jira issue key. On GitHub, verify the branch and commit. In Jira, connect the GitHub integration/app if required, then check the issue's Development section.

Experiment 3: Git Commands and Version Control

Fork node_test if you have not already done so. Clone your own fork:

```
git clone https://github.com/YOUR_USERNAME/node_test.git
cd node_test
```

```
git checkout -b experiment-1
git branch
```

Create practical.txt and enter this exact content

```
nano practical.txt
```

```
ASDD Practical Experiment 3
Git and GitHub Version Control
Name: YOUR_NAME
Purpose: Practising Git commands using Ubuntu Terminal.
This file is created, staged, committed and pushed to GitHub.
```

Save (Ctrl+O → Enter → Ctrl+X), then execute:

```
git status
git add practical.txt
git status
git commit -m "Added practical file"
git push -u origin experiment-1
git log --oneline
```

Open your GitHub fork, select the experiment-1 branch and confirm practical.txt is visible.

Experiment 4: Docker Image Management and Container Execution

Install and start Docker

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
docker --version
```

Pull and inspect the Ubuntu image

```
sudo docker pull ubuntu
sudo docker images
```

Run an interactive Ubuntu container

```
sudo docker run -it ubuntu bash
```

Inside the container, type these commands:

```
ls
cat /etc/os-release
exit
```

Run a container in the background and verify

```
sudo docker run -dit --name mycontainer ubuntu bash
sudo docker ps
sudo docker ps -a
```

Optional cleanup after verification:

```
sudo docker stop mycontainer
sudo docker rm mycontainer
```

Experiment 5: Dockerfile Creation and Image Building

Create a project folder:

```
mkdir docker-practical
cd docker-practical
```

Create index.html — exact content

```
nano index.html
```

```
<!DOCTYPE html>
<html>
<head>
    <title>Docker Practical</title>
</head>
<body>
    <h1>Hello from Docker!</h1>
    <p>ASDD Practical Experiment 5</p>
</body>
</html>
```

Save and exit nano.

Create Dockerfile — exact content

```
nano Dockerfile
```

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Save and exit nano. Build the image:

```
sudo docker build -t mywebapp .
```

Run and verify the container:

```
sudo docker images
sudo docker run -d -p 8080:80 --name webcontainer mywebapp
sudo docker ps
```

Open http://localhost:8080 in your browser. Expected page text: “Hello from Docker!” and “ASDD Practical Experiment 5”.

Optional cleanup:

```
sudo docker stop webcontainer
sudo docker rm webcontainer
```

Experiment 6: Terraform and GitHub Actions

Fork https://github.com/shruthi1722/terraform23 in GitHub. Clone your fork:

```
git clone https://github.com/YOUR_USERNAME/terraform23.git
cd terraform23
ls
ls *.tf
```

Edit main.tf

```
nano main.tf
```

Important: inspect the existing file first. Keep its current provider/resource configuration. Make the change requested by the examiner. If a simple output block is suitable, append this exact block at the end:

```
output "message" {
  value = "Hello from Terraform"
}
```

Save and exit. Create a branch and push the change:

```
git checkout -b terraform-update
git status
git add main.tf
git commit -m "Updated Terraform configuration"
git push -u origin terraform-update
```

Verify GitHub Actions

14. Open your fork on GitHub.
15. Click the Actions tab.
16. Open the workflow run triggered by your push, if one is configured for that branch/event.
17. Open the job and check the step results. A green check means the workflow succeeded.

If no workflow runs, inspect the repository's .github/workflows folder and its trigger settings. If Terraform CLI is installed and the practical asks you to validate the file, use:

```
terraform --version
terraform init
terraform fmt
terraform validate
terraform plan
```

Do not run terraform apply or terraform destroy unless the examiner specifically asks and you understand the infrastructure changes.

Experiment 7: Grafana Using Docker

Pull and run Grafana

```
sudo docker pull grafana/grafana
sudo docker images
sudo docker run -d -p 3001:3000 --name grafana grafana/grafana
```

Verify and open dashboard

```
sudo docker ps
```

Open http://localhost:3001 in your browser. The default initial login is commonly:

```
Username: admin
```

```
Password: admin
```

Grafana may ask you to change the password on first login.

If a container named grafana already exists, do not create another with the same name. Check and start it:

```
sudo docker ps -a
sudo docker start grafana
```

Optional cleanup:

```
sudo docker stop grafana
sudo docker rm grafana
```

Quick Command Revision

Command	Purpose
git clone URL	Clone a repository
git checkout -b branch	Create and switch branch
git branch	List branches
git status	Check changes
git add .	Stage changes
git commit -m "message"	Commit changes
git push -u origin branch	Push branch
docker pull IMAGE	Download image
docker images	List images
docker run ...	Run a container
docker ps	List running containers
docker ps -a	List all containers
docker build -t NAME .	Build image from Dockerfile
terraform init	Initialize Terraform
terraform validate	Validate configuration
terraform plan	Preview infrastructure changes

END OF PRACTICAL GUIDE